

Day 2 Submission

Name: Gourik Narendra Patil

College: Vidyalankar Polytechnic

Branch / Year: CO5KC

Variant ID: D2-C

Contents

Day 2 Submission.....	1
1. Founder Assumption.....	2
2. Competing Hypotheses.....	2
3. Origin Failure.....	3
4. Symptom vs Root Cause Table	3
5. Failure Propagation Chain.....	4
6. First 15-Minute Incident Actions	4
7. What I Would Not Fix Immediately.....	4
8. Tradeoff Ownership.....	5
9. AI / Assumption Challenge.....	5
10. Final Incident Judgment	5
One-line realization	5
Failure Propagation Analysis	6
1. Origin Failure.....	6
2. Propagation Chain.....	6
3. Symptom Separation.....	7
4. Business Damage	7
5. Final Propagation Judgment	7
Incident Prioritization Artifact.....	8
1. First Stabilization Priority	8
2. Delay List.....	8
3. First 15-Minute Plan.....	8
4. Stakeholder Impact.....	9
5. Final Priority Defense	9

1. Founder Assumption

The founder assumed the issue is caused by:

- dashboard bugs OR
- notification/recruiter action issues

Judgment: Misleading

Why:

- Evidence shows DB writes are successful (~99% success, normal latency range)
- Notifications eventually send correct status
- Mismatch originates before dashboard/notifications — at cache layer

2. Competing Hypotheses

Hypothesis	Evidence Supporting It	Evidence Against It	Decision
DB write failure	Slight latency increase	99% success rate, logs show successful commits	Rejected
Notification system wrong	Delays exist	Notifications eventually send correct status	Rejected
Cache invalidation failure	18.5% mismatch, invalidation failures, logs show stale cache	Strong direct evidence	Accepted

3. Origin Failure

Root Cause: Redis cache invalidation failure under high load

Confidence: High

Why:

- E1: `WARN Redis invalidation failed`
- E2: Cache mismatch rate jumped from `<0.5% → 18.5%`
- E3: `ERROR Cache invalidation batch failure reason=redis_timeout`

4. Symptom vs Root Cause Table

Reported Issue	Root Cause / Downstream Effect	Evidence
Recruiter actions not updating	Downstream (stale cache reads)	DB commit success logs
Duplicate shortlist actions	Downstream (UI sees stale state)	Cache mismatch
Notification delays	Independent but secondary	Queue delay increased
Candidate status mismatch	Direct effect of cache failure	cache vs db mismatch logs
Dashboard lag	Downstream	reads from cache + delay
Analytics inconsistency	Downstream	analytics uses dashboard snapshot

5. Failure Propagation Chain

Origin failure:

Cache invalidation failure (Redis timeout under spike)

- Cache not updated after DB write
- Dashboard reads stale cache
- Recruiters see wrong status
- Duplicate actions triggered
- Candidate portal shows stale status
- Notifications conflict with UI
- Analytics polluted from dashboard snapshot

→ Business impact:

- Wrong decisions
- Duplicate shortlist risk
- Loss of trust

6. First 15-Minute Incident Actions

Action	Why Now	Risk Reduced	Tradeoff
Disable Redis reads / force DB reads	Removes stale data immediately	Stops incorrect decisions	Slower dashboard
Pause cache invalidation batch jobs	Prevents further corruption	Stabilizes state	Cache freshness reduced
Alert recruiters to slow actions	Prevent duplicate actions	Reduces data inconsistency	Lower throughput

7. What I Would Not Fix Immediately

Delayed Work	Why Delay	Risk Accepted
Notification delays	Not source of truth	Slight delay in communication
Analytics correction	Non-critical	Temporary reporting errors
Dashboard UI fixes	Not root cause	UI inconsistency remains briefly

8. Tradeoff Ownership

Tradeoff: Prioritize correctness over speed

- Helped: Recruiters, candidates, hiring decisions
 - Impacted: Dashboard speed, analytics freshness
-

9. AI / Assumption Challenge

Initial instinct / founder assumption:

- Dashboard or notification issue

Evidence showed:

- DB writes are correct
- Notifications eventually correct
- Cache is serving stale data

What changed my mind:

- Cache mismatch rate spike (18.5%)
 - Explicit invalidation failures in logs
-

10. Final Incident Judgment

A Redis cache invalidation failure under high traffic caused stale candidate states to persist. This poisoned dashboard reads, recruiter decisions, candidate views, and analytics.

First priority: restore source-of-truth consistency by bypassing cache.

One-line realization

I initially thought the dashboard or notifications were broken, but the evidence showed stale cache was poisoning the entire system.

Failure Propagation Analysis

Student: Gourik Narendra Patil
Variant ID: D2-C

1. Origin Failure

Redis cache invalidation failure

Evidence:

- 18.5% mismatch rate (vs <0.5% normal)
- 312 invalidation failures
- `redis_timeout` errors
- Logs show DB updated but cache not invalidated

Confidence: High

2. Propagation Chain

Origin failure:

Cache invalidation failure

- Cache retains stale candidate status
- Dashboard reads stale cache
- Recruiters see incorrect status
- Recruiters repeat actions
- Candidate portal serves stale data
- Notifications conflict with UI
- Analytics records incorrect snapshot

→ Business impact:

- Duplicate actions
 - Candidate confusion
 - Wrong shortlist numbers
-

3. Symptom Separation

Symptom	Direct Cause	Downstream From	Evidence
Status mismatch	stale cache	invalidation failure	logs
Dashboard inconsistency	cache read	stale state	mismatch logs
Duplicate actions	wrong UI state	stale cache	recruiter complaints
Analytics mismatch	dashboard snapshot	stale cache	analytics logs

4. Business Damage

First affected: Recruiters

- Making wrong decisions

Then: Candidates

- Seeing inconsistent status

Then: Hiring ops

- Wrong shortlist numbers

Evidence:

- Dashboard mismatch 21%
 - Recruiter complaints
 - Candidate complaints
-

5. Final Propagation Judgment

A cache invalidation failure allowed stale data to persist after correct DB writes. All read-heavy systems (dashboard, portal, analytics) consumed this stale state, creating widespread inconsistency and decision errors.

Incident Prioritization Artifact

Student: Gourik Narendra PAtil
Variant ID: D2-C

1. First Stabilization Priority

Disable cache reads / bypass Redis

Why:

- Cache is serving incorrect data
- DB is still reliable source of truth

Evidence:

- Cache mismatch 18.5%
- DB commit success ~99%
- Logs show stale cache reads

2. Delay List

Work To Delay	Why Delay It	Risk Accepted
Notification fixes	Not root cause	Delay in updates
Analytics fixes	Non-critical	Temporary wrong reports
UI fixes	Cosmetic vs core issue	Visible inconsistency

3. First 15-Minute Plan

Minute Window	Action	Risk Reduced	Tradeoff
0-5	Disable cache reads	Stops stale data	Increased latency
5-10	Pause invalidation jobs	Prevents further errors	Cache unusable
10-15	Communicate to recruiters	Prevent duplicate actions	Slower workflow

4. Stakeholder Impact

Benefited:

- Recruiters (correct decisions)
- Candidates (correct status)

Still impacted:

- Dashboard users (slower)
 - Analytics users (temporary mismatch)
-

5. Final Priority Defense

Fixing the cache first prevents incorrect decisions and data corruption.
Dashboard or notification fixes would only mask symptoms.
Correctness is critical due to hiring deadline constraints.
Speed degradation is acceptable; incorrect decisions are not.

GNP-29